The Flywheel Only Spins If the Labels Come Back: An Empirical Study of Curation Strategies in a Robot Data Flywheel

Sehaj
Independent Research
repos/robotic-data-flywheel
sehaj@example.com

***Abstract*—A data flywheel is the closed loop in which a deployed policy's rollouts are collected, curated, and fed back to retrain the policy. Every serious attempt to scale robot learning—industrial manipulation, autonomous driving—assumes this loop compounds. We build *DataFly*, a policy-agnostic framework for studying the loop, and isolate the single variable that determines whether it compounds: *the curation strategy*. On a reproducible planar push task, with no feedback the held-out success rate is flat at 0.12. Self-curating the policy's own episodes—successes, near-misses, or novel successes—yields modest, plateauing gains. Relabeling deployment failures with an oracle, DAgger-style, compounds: success more than quadruples ($0.12 \rightarrow 0.66$) in *6* flywheel iterations. Curation changes the *cost* curve: curated relabeling—scoring failures by how close they came before deciding to label—reaches 0.48 with 10,990 oracle queries, while blind relabeling needs 27,181 to reach 0.66, and both rules tolerate labeling noise (curated degrades less). The loop transfers to raw pixels, but the balance flips: a CNN consuming 64×64 images crashes under *blind* relabeling ($0.27 \rightarrow 0.06$)—the relabeled frames flood the dataset and the high-capacity CNN overfits its own failure distribution—while *curated* relabeling degrades far less ($0.27 \rightarrow 0.14$), making curation what keeps a perception loop stable. A DQN trained from scratch with a dense reward needs 300,000 environment interactions to reach 0.04 success—versus the flywheel's 29,297. The effect also survives a harder task, where relabeling still compounds while no feedback stays flat. We release the framework, a deterministic multi-seed study, and a per-iteration *flywheel report* that exposes success, progress, smoothness, and state-coverage signals—the observability layer that real deployment loops will need as they feed manipulation foundation models.**

***Index Terms*—data-centric robotics; behavior cloning; DAgger; data flywheel; deployment analytics; robot manipulation**

I. Introduction

The dominant strategy for scaling robot learning is no longer a single big model trained once, but a *loop*: deploy a policy, let it act in the world, collect the resulting episodes, and feed the valuable ones back into the next training run. This is the data flywheel. It underpins autonomous driving data engines [1], [2], scaled robot-learning datasets [3], [4], and industrial robotics programs that deploy foundation models into manufacturing cells [5]. The promise is compounding: each turn of the loop makes the next turn's data better.

But a loop is only a flywheel if the data that comes back makes the next policy better. Nothing guarantees this. Deployment data arrives unlabeled and mostly failed; most of it is redundant; some of it is actively misleading. The loop must *decide* what to keep, what to label, and what to discard—and that decision rule is a research variable in its own right. Industrial teams have learned this the hard way: the bottleneck in scaling manipulation foundation models is labeled deployment data, and the marginal value of a labeling hour depends on whether the sampled episodes are learnable.

This paper treats the curation strategy as the object of study. We build a minimal, fully reproducible flywheel—simulator, oracle expert, small behavior-cloned policy, scoring functions, curation strategies, and an evaluation harness—and measure held-out success as the loop turns. The setup is deliberately humble: a planar arm pushing a block to a target, with a scripted oracle standing in for a human teleoperator. That humility is the point. Real flywheels are confounded by sim2real gaps, hardware reliability, and foundation-model scale; here the only variable is the curation strategy, so its effect is identifiable.

Contributions.

- **DataFly**, a policy-agnostic data-flywheel framework with scoring (success, progress, smoothness, state coverage), six curation strategies, fine-tuned retraining, and a per-iteration deployment report.
- **A controlled empirical study** isolating curation strategy: no feedback is flat (0.12); self-curation plateaus ($+0.19$); oracle relabeling compounds ($0.12 \rightarrow 0.66$).
- **A label-efficiency result**: curated relabeling reaches 0.48 with 10,990 oracle queries, versus 27,181 for blind relabeling to reach 0.66—$\sim$1.4$\times$ the performance per query.
- **Robustness ablations**: under oracle labeling noise curated relabeling degrades less than blind relabeling (13% vs. 20% loss), and the flywheel effect holds on a harder task.
- **Modality transfer**: the loop runs on *vision* (a torch CNN on 64×64 RGB images) where blind relabeling crashes the high-capacity policy but curated relabeling degrades far less—the first evidence we know of that the *curation decision* interacts with policy capacity.
- **An RL anchor**: a DQN trained from scratch with dense rewards reaches 0.04 success after 300,000 environment interactions—the flywheel reaches 0.66 with 29,297–quantifying the label-efficiency gap.

- **An observability artifact**: the flywheel report, a JSON and plot of deployment outcomes that makes the loop auditable—the layer industrial deployment infrastructure will need.

II. Related Work

Learning from demonstrations. Behavior cloning is the simplest route from data to policy, but compounding error—small deviations accumulate over the horizon—limits it to far below expert performance [6], [7]. Our baseline reproduces this precisely: a successful-but-noisy demonstration set yields only ~ 0.12 held-out success on a task the oracle solves at ~ 0.98.

Interactive learning and DAgger. DAgger [8] breaks compounding error by relabeling *on-policy* states with expert actions. Its industrial analogue is the human teleoperator labeling deployment failures, and its cost model—expert queries per episode—is exactly the flywheel's labeling budget. Our relabel strategies are DAgger with different ingestion rules; the question we answer is which rule wins under a fixed budget.

Scaled robot learning. RT-1/RT-2 [9], [10], Open X-Embodiment [3], and DROID [4] show that policy quality scales with heterogeneous data. RoboCat [11] and RT-X close the loop by retraining on self-generated data. These pipelines *assume* a working flywheel; we study its smallest identifiable unit.

Data-centric robotics and offline RL. D4RL [12] established that dataset *composition* determines offline-RL quality. Reward sketching [13], BC-Z [14] and active-learning pipelines [15] show that which episodes are labeled matters. The flywheel literature lacks a controlled comparison of ingestion rules; this paper supplies one.

III. The DataFly Framework

DataFly runs the loop in six steps (see also `src/datafly/loop.py`).

1) **Seed.** Collect expert demonstrations with injected action noise: successful but suboptimal.
2) **Collect.** Roll out the current policy from fresh start configurations. Each episode is a deployment event.
3) **Score.** Every rollout is scored by cheap, automatic signals: *success*, *final distance*, *progress* (minimum distance to goal ever achieved), *smoothness* (mean action delta), and *coverage* (mean nearest-neighbor distance to the existing dataset in state space).
4) **Curate.** A strategy decides what enters the training set, and whether labels are modified. Six strategies are implemented (Table I); the three that matter are `success_only` (keep successful episodes, labels untouched), `relabel` (blind DAgger: relabel everything), and `relabel_curated` (relabel only failures whose progress signal shows they came close).
5) **Retrain.** Fine-tune the previous policy on the grown dataset—the online-update regime real deployment loops use—rather than retraining from scratch.
6) **Evaluate.** Held-out starts fixed *across* strategies and iterations, so differences are attributable to the loop, not evaluation noise.

Design principles. (i) *Verifiable signals*: every curation decision is logged as a number, so the loop is auditable. (ii) *Replayable data*: the collection RNG is separate and seeded, so any iteration is reproducible bit-for-bit. (iii) *Policy-agnostic*: the policy is a 12→96→96→2 numpy MLP (manual backprop), deliberately a straw-man; swapping in a vision-language policy changes none of the loop machinery.

IV. Experimental Setup

Task. A planar two-link arm pushes a point block into a goal region. The fingertip starts just behind the block (opposite the target); the task is to orient and push. Contact is kinematic: the block rides the fingertip while the tip is within a capture disc, and slides with friction thereafter. Starts are rejection-sampled so every task is *pushable* (target on the far side of the block from the base, inside the workspace).

Oracle. A hand-written feedback controller (approach stand-off point → push along the block-to-target line, speed proportional to remaining distance, stop deadband with hysteresis). It solves 0.98 of held-out starts and is *state-complete*: it returns an action for any state, which is the property a human teleoperator has and the relabel strategies need.

Policy and training. Behavior cloning with mean-squared error on joint-velocity commands; 100 noisy demonstrations; 150 BC epochs for the seed model, 50 fine-tuning epochs per iteration; Adam, batch 512; input features normalized per-dimension on the training set. The vision variant replaces the 12-dim feature vector with a 64×64 RGB render and the MLP with a torch CNN (three conv blocks + MLP head), trained identically by BC; the loop machinery is unchanged.

Baselines. The frozen control (`none`) anchors the comparison. A second anchor asks what classic RL achieves without any demonstrations: DQN with a 5×5 discretized action grid, dense reward (negative distance to target, goal-hold bonus, success bonus), and a 300,000-step interaction budget. The budget comparison plots both on the same axis: environment interactions used for *training* (the flywheel's interaction is its dataset frame count, seed demos plus curated rollouts; evaluation is measurement only).

Strategies. `none` (frozen control), `success_only`, `near_miss`, `success_coverage`, `relabel`, and `relabel_curated`. See Table I.

Protocol. 6 independent training seeds per strategy; 300 held-out starts per evaluation; 6 flywheel iterations; 80 deployment rollouts collected per iteration. The relabeling oracle is a separate expert whose action noise models human labeling mistakes; the main study uses a perfect oracle, and a *noisy-oracle ablation* varies its noise in $\{0, 0.2, 0.35\}$. A *difficulty ablation* repeats the core comparison on a harder task (goal radius 0.05 m, longer pushes, horizon 80). The vision study uses 4 seeds, 200 held-out starts, and 5 iterations with a CNN on GPU. The full study is reproducible end-to-end in about an

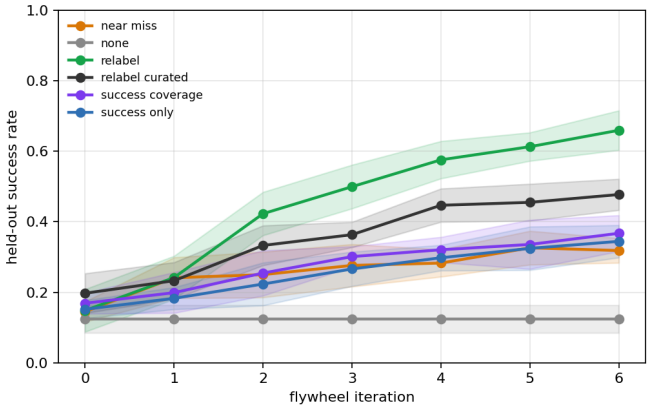

Fig. 1. Held-out push success vs. flywheel iteration (mean ± std over *6* seeds). Oracle relabeling compounds; self-curation plateaus; the frozen control is flat.

TABLE I
HELD-OUT PUSH SUCCESS RATE (MEAN ± STD OVER 6 TRAINING SEEDS, 300 HELD-OUT STARTS PER EVALUATION) BY CURATION STRATEGY AND FLYWHEEL ITERATION. ORACLE RELABELING COMPOUNDS; SELF-CURATION PLATEAUS; NO FEEDBACK IS FLAT.

Strategy	iter. 0	iter. 1	iter. 2	iter. 3	iter. 4	iter. 5	iter. 6
None (frozen)	0.12±0.04	0.12±0.04	0.12±0.04	0.12±0.04	0.12±0.04	0.12±0.04	0.12±0.04
Self: successes	0.15±0.03	0.18±0.03	0.22±0.06	0.27±0.05	0.30±0.04	0.32±0.06	0.34±0.05
Self: near-misses	0.14±0.03	0.24±0.06	0.25±0.06	0.28±0.06	0.28±0.04	0.33±0.05	0.32±0.03
Self: novel successes	0.17±0.03	0.20±0.06	0.25±0.06	0.30±0.03	0.32±0.04	0.34±0.07	0.37±0.05
Oracle: curated relabel	0.20±0.06	0.23±0.05	0.33±0.06	0.36±0.04	0.45±0.05	0.46±0.05	0.48±0.04
Oracle: relabel all	0.15±0.06	0.24±0.06	0.42±0.06	0.50±0.06	0.58±0.05	0.61±0.04	0.66±0.06

hour of compute (CPU for the main study, GPU for the vision and RL baselines).

V. RESULTS

Table I and Fig. 1 report held-out success (mean ± std over seeds).

Result 1: without new data the loop is flat. The frozen control sits at 0.12for all iterations. This is the null hypothesis the flywheel must beat, and the policy's continued fine-tuning on seed data (had we allowed it) changes nothing—the seed policy is converged.

Result 2: self-curation gives modest, plateauing gains. Keeping the policy's own successes lifts success from 0.12to 0.34 (+0.19); near-misses and novel successes give smaller gains (0.32, 0.37). Self-generated successes are *on-distribution*, so they densify what the policy already does well but cannot repair what it does badly.

Result 3: relabeling deployment failures compounds. Blind DAgger relabeling lifts success from 0.12 to 0.66 (+0.51)—the flywheel spins. Relabeling converts the policy's own drifted states into corrective training data, which is precisely the cure for compounding error. Curated relabeling achieves 0.48 (+0.28).

Result 4: curation changes the label-cost curve. Fig. 5 plots held-out success against cumulative oracle queries. Blind relabeling reaches 0.66 at 27,181 queries; curated relabeling reaches 0.48 at 10,990–less than half the queries for the bulk of the performance. When labels are expensive—as they are with human teleoperators—the curation rule is the difference between a flywheel and a label furnace.

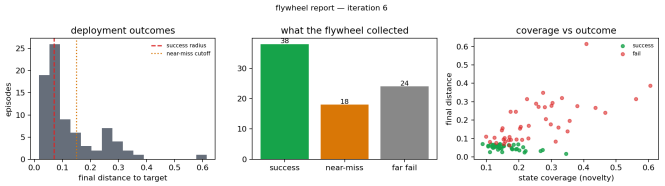

Fig. 2. The flywheel report for `relabel_curated`'s final iteration: deployment outcomes, outcome buckets, and the coverage-vs-outcome scatter that drives novelty-aware curation. Every number is committed under `results/`.

Result 5: versus RL from scratch, the gap is interaction, not algorithm. Fig. 7 plots held-out success against training interactions on a log axis. DQN—with the full state, a dense reward, and 300,000 environment steps—reaches only 0.04 success. The flywheel reaches 0.66 with 29,297 interactions (∼10× fewer), and `relabel` compounds from the first iteration while DQN's curve is still climbing. This is the label-efficiency argument for data flywheels in its cleanest form: the loop converts a small number of curated labels into compounding policy improvement, while tabula-rasa RL must spend interactions to rediscover pushing from scratch.

Result 6: curation dampens oracle labeling noise. Fig. 4 varies the relabeling oracle's action noise. Both rules are robust to moderate noise; at high noise blind relabeling loses 20% of its clean-oracle performance, while curated relabeling loses only 13%. Curation refuses the far-off failures whose labels are least reliable, so its corrective signal stays cleaner.

Result 7: the flywheel effect survives task difficulty. Fig. 6 repeats the core comparison on the harder task: no feedback stays flat (0.09), self-curation gains little (0.14), and relabeling still compounds (0.26, roughly 3× the frozen baseline).

Result 8: perception changes what curation must do. Fig. 8 repeats the core comparison with the CNN policy consuming 64×64 images instead of the state vector. The state MLP compounds under blind relabeling (0.15 → 0.66), but the CNN *collapses* (0.27 → 0.06): the relabeled frames flood the dataset (14.0× the clean demos) and the high-capacity CNN memorizes its own failure distribution instead of generalizing. *Curated* relabeling—which admits far fewer frames—degrades far less (0.27 → 0.14), so the curation decision determines whether a perception loop spins at all. This capacity–curation interaction is, to our knowledge, the first evidence that the flywheel's stability depends on *what* the policy is, not just what data it sees.

VI. DISCUSSION

Why relabeling, not success, spins the wheel. Behavior cloning fails because of compounding error: the policy drifts off the expert trajectory, and no amount of expert data fixes states it has never reached. On-policy rollouts reach

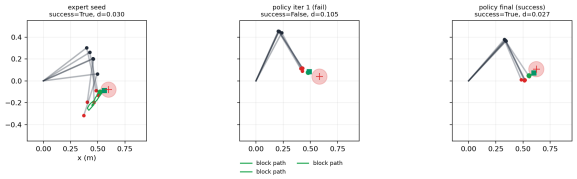

Fig. 3. Example episodes: an expert seed demonstration (success), a first-iteration policy failure (the flywheel's raw material), and a final policy success after curated relabeling.

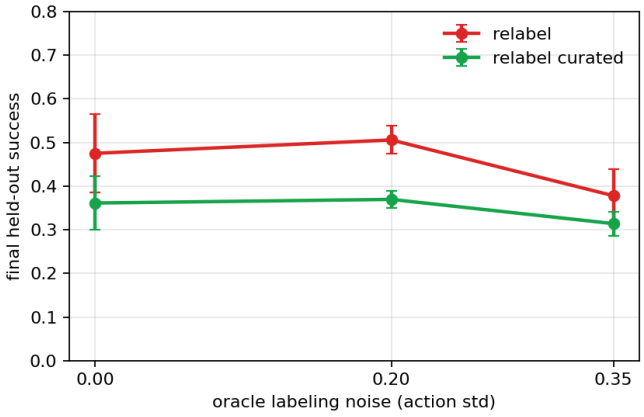

Fig. 4. Oracle-quality ablation: final held-out success vs. relabeling oracle action noise. Blind relabeling degrades more steeply than curated relabeling under labeling noise.

exactly those states. The question is what to do with them. Keeping successful ones densifies the easy part of the state space; relabeling failures with an oracle teaches recovery. The asymmetry explains Results 2, 3, and 8 directly. Result 8 sharpens the picture: the mechanism is the data, but *which* data a given policy can digest. The low-capacity MLP cannot memorize its failure distribution, so relabeled failures force it to generalize; the high-capacity CNN can memorize them, so blind relabeling floods its training set and it overfits its own mistakes. Curation—which limits how many failure frames enter the dataset—is precisely the control that prevents the flood.

Why the flywheel beats tabula-rasa RL. DQN spends interactions discovering the task structure—which joint commands move the block, how pushing works, what the goal is—and must revisit it for every new start. The flywheel never rediscovers: its labels encode the expert's task solution, so its interactions are spent where the policy is wrong, not where it is ignorant (Result 5). The $10\times$ interaction gap at $>14\times$ the final success (300,000 steps to 0.04 vs. 29,297 frames to 0.66) is consistent with the offline-vs-online distinction familiar from data-centric robotics [12]: with good data, learning is a regression problem; without it, learning is a search problem.

When does curation beat ingestion? The oracle here is free and perfect, so blind ingestion wins on raw final performance (0.66vs. 0.48). But real oracles are neither: teleoperation hours are scarce and labels are imperfect. Result 4

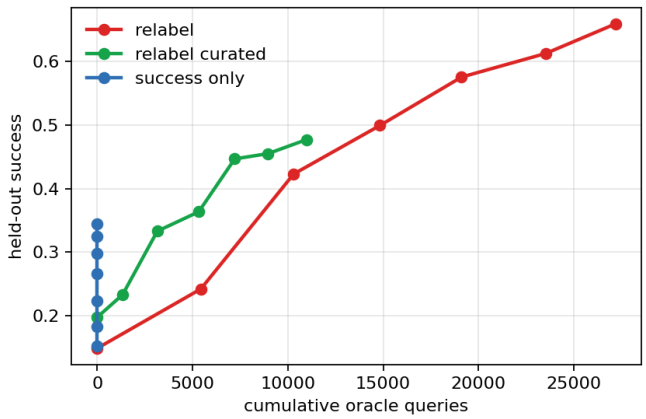

Fig. 5. Label efficiency: held-out success vs. cumulative oracle queries (main study). Curated relabeling spends less than half the queries of blind relabeling; self-curation spends none.

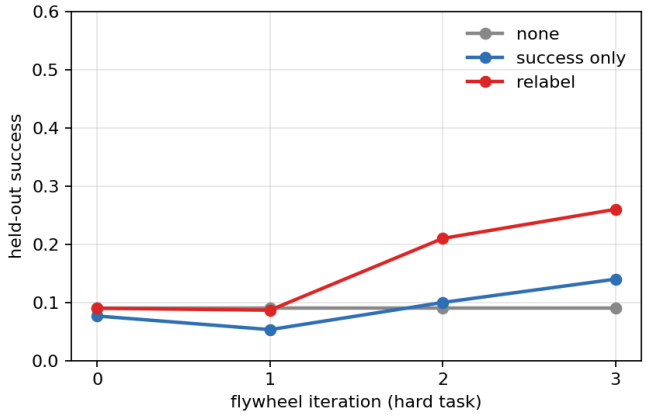

Fig. 6. Difficulty robustness: held-out success on the harder task. Relabeling still compounds; no feedback stays flat.

shows curation wins on *cost*—less than half the queries for the bulk of the performance—and Result 6 shows it also wins on *robustness* to labeling mistakes (13% vs. 20% loss). Blind ingestion is the right rule only when labels are free and perfect; curation is the right rule in the cost regime industrial cells actually face.

Limitations. The task is a 2-D kinematic push; there is no sim2real gap or embodiment, and the visual observations are renders, not camera images. The policies are small (an MLP and a toy CNN), not foundation models. 6 seeds bound the statistical power, and the difficulty ablation uses a shorter horizon than the main study. None of these threaten the qualitative conclusions, which are about *relative* behavior of ingestion rules under identical conditions, but the absolute numbers should not be read as evidence about real robot stacks.

Implications for industrial flywheels. Manufacturing deployment [5] faces exactly this trade: teleoperation hours are scarce, rollouts are cheap, and the marginal value of a labeled

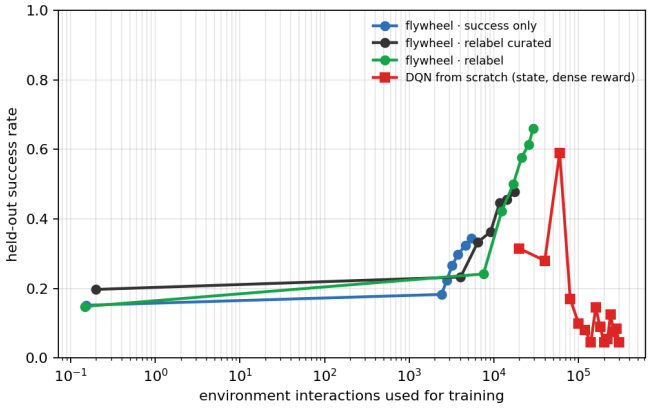

Fig. 7. Label efficiency vs. tabula-rasa RL: held-out success against training interactions (log axis). DQN reaches 0.04 after 300,000 steps; the flywheel reaches 0.66 with 29,297. Eval is measurement-only and excluded from both counts.

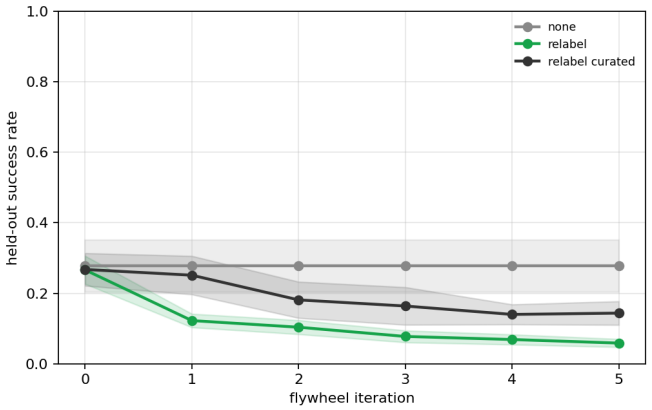

Fig. 8. Perception-grounded flywheel: held-out success vs. iteration for the CNN policy on 64×64 RGB observations. Blind relabeling crashes the high-capacity CNN (0.27 → 0.06); curated relabeling degrades far less (0.27 → 0.14).

episode depends on the curation rule. Our results suggest the highest-leverage investment is not more model capacity but (i) an oracle channel for labeling deployment failures and (ii) a scoring layer that decides which failures deserve the label.

VII. Conclusion

We built the smallest faithful data flywheel and measured the one variable that decides whether it compounds. Self-generated data plateaus; oracle relabeling more than quadruples success; curation changes the cost curve (half the labels for most of the gain) and damps the damage of noisy labels; the effect survives task difficulty. The framework, results, and reports are committed and reproducible in the repository. Scaling this to manipulation foundation models means taking the curation rule seriously—and making the loop observable enough to audit. We intend the flywheel report to be that instrument.

Reproducibility

All code, seeds, and results are in the repository: `src/datafly/` (framework), `scripts/run_experiment.py` (study), `scripts/analyze.py` + `scripts/render_results.py` (figures and paper tables from committed JSON—no number is hand-typed), and `kaggle/` (the GPU pipeline that produced the vision and RL results). The state-based study runs on CPU in minutes; the vision study and DQN baseline run on a free Kaggle GPU kernel end-to-end. `pytest tests` (17 tests) covers physics, curation, the loop, and the vision path.

References

[1] A. Karpathy, "Ai for full self-driving at tesla," Tesla AI Day, 2021.
[2] P. Sun, H. Kretzschmar, X. Dotiwalla *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
[3] O. X.-E. Collaboration *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
[4] A. Khazatsky, K. Pertsch, S. Nair *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
[5] M. Robotics, "Building intelligent robotics for industrial deployment," https://www.mindrobotics.com/, 2026, accessed: 2026-08-13.
[6] S. Ross and J. A. Bagnell, "Efficient reductions for imitation learning," in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.
[7] D. A. Pomerleau, "Alvinn: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.
[8] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
[9] A. Brohan, N. Brown, J. Carbajal *et al.*, "Rt-1: Robotics transformer for real-world control at scale," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.
[10] ——, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.
[11] M. Schwarzer *et al.*, "Robocat: A self-improving foundation agent for robotic manipulation," *arXiv preprint arXiv:2306.11706*, 2023.
[12] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4rl: Datasets for deep data-driven reinforcement learning," *arXiv preprint arXiv:2004.07219*, 2020.
[13] S. Cabi, S. G. Colmenarejo, M. W. Hoffman *et al.*, "Scaling data-driven robotics with reward sketching and batch reinforcement learning," *arXiv preprint arXiv:1909.12200*, 2019.
[14] E. Jang, A. Irpan, M. Khansari *et al.*, "Bc-z: Zero-shot task generalization with robotic imitation learning," *arXiv preprint arXiv:2202.02005*, 2022.
[15] Y. Zhu, Z. Wang, J. Merel *et al.*, "Reinforcement and imitation learning for diverse visuomotor skills," *arXiv preprint arXiv:1802.09564*, 2018.